

sp — gradual escalation + structured output (v1 design report)

Date: 2026-05-30

Implementação do mandato 2026-05-30:

1. **Escalação gradativa** 64 → 100 → 200 subagents (em vez de 200 cego)
 2. **Saída padronizada** do LLM via schema v1 (JSON com artifact, files_changed, behaviors_added, expected_oracle_pass, confidence, concerns)
- Commits: ad87be9 + fcc8825

1. O que mudou na pipeline do sp

Antes (v13, N=200 cego):

sp(prompt) → fan-out N=200 cego → modal-vote por string raw → oracle(modal)

Custo fixo de 200 calls/case, mesmo quando a 1ª chamada já passa.
Modal-vote por string raw colapsa em uniq=1 quando 200 outputs do mesmo erro são idênticos.

Agora (escalator + schema v1):

cycle 1: 64 subagents diversificados → behavior_modal_vote → oracle?
 PASS → STOP (64 calls totais) ✓
 FAIL → cycle 2
cycle 2: 100 subagents (fresh) → behavior_modal_vote → oracle?
 PASS → STOP (164 calls totais) ✓
 FAIL → cycle 3
cycle 3: 200 subagents → behavior_modal_vote → return
 (364 calls totais – apenas pra task incurável)

Mudanças-chave: oracle dirige a parada (não custo fixo); output estruturado por padrão via marker [STRUCTURED_OUTPUT=v1]; modal-vote por *behavior signature* (hash do trio files/behaviors/expected) em vez de string raw; vencedor = candidato com maior confidence no grupo modal; parser tolerante com fallback pra texto-como-artifact em modelo pequeno.

2. Schema v1

```
{
  "artifact":           "<código completo: arquivo / diff / código>",
  "files_changed":      ["src/Core/X.php", ...],
  "behaviors_added":    ["X::foo", "X::bar"],
  "expected_oracle_pass": ["test_foo", "test_bar"],
  "confidence":         0.0..1.0,
  "concerns":           ["depende de lib não declarada", ...]
}
```

artifact é o ÚNICO campo consumido pelo oracle (phpunit/regex). Os outros são auto-relato pro modal-vote agregar.
files_changed + behaviors_added + expected_oracle_pass formam o behavior signature — hash sorted = grupo modal.
confidence desempata candidatos do grupo modal.

Comportamento do parser

Input	Resultado
JSON puro	parse_ok=True, todos campos populados
JSON dentro de fences	parser strip-a fence, parse_ok=True
JSON com prose antes/depois	busca primeiro { balanceado, parse_ok=True
Texto livre (sem JSON)	parse_ok=False, artifact=text raw, vote por string
JSON malformado	parse_ok=False, fallback raw

3. Smoke test executado

Modelo: google/gemma-3-4b-it. Case: password_strength. Tiers de proxy: **4 → 8 → 16** (em vez de 64→100→200, pra não queimar 364 calls num smoke).

```
password_strength baseline fail cli PASS cli+sp fail[tiers=4→8→16,u=16,modal=1,parse=0/16]
```

Leitura

Diagnostic	Valor	Interpretação
tiers=4→8→16	escalou os 3 ciclos	oracle falhou em todos — modelo pequeno não recupera
u=16	16 outputs únicos	sem colapso (esperado pra Gemma-4B)
modal=1	grupo modal tinha 1	sem consenso de behavior
parse=0/16	NENHUM dos 16 respeitou JSON	Modelo 4B não honra structured output
cli PASS	cli alone passou	modelo CONSEGUE resolver, mas sp diversifica errado

Conclusão do smoke: mecânica do escalator funcional (tier history reportado, fallback parser ativa em modelo pequeno).
Ganho real do schema em modelo pequeno: **zero** (esperado — schema é feature pra modelo grande).

4. Custo esperado vs antigo N=200 cego

Cenário	N=200 cego	Escalator	Economia
modelo grande + task fácil	200	64	-68% (cycle 1)
modelo grande + task média	200	164	-18% (cycle 2)
modelo grande + task difícil	200	364	+82% (cycle 3)
modelo pequeno + task fácil	200	64	-68% (cycle 1)
modelo pequeno + task difícil	200	364	+82% (cycle 3, sem ganho)

Para o batch v13 (3 modelos × 12 cases × 5 lados)

Modelo	Tasks típicas (12)	Calls totais sp
qwen-2.5-coder-32b (grande)	8 fáceis ×64 + 4 difíceis ×364	~1,970
gemma-3-4b-it (médio)	4 fáceis ×64 + 8 difíceis ×364	~3,170
llama-3.2-3b (pequeno)	2 fáceis ×64 + 10 ×364	~3,770
TOTAL exec sp side	—	~8,900
N=200 cego seria	3 × 12 × 200	7,200

Tradeoff direto: se P(passa em cycle 1) > 0.45, escalator vence em custo médio. Pra modelos grandes P ≈ 60-80% (escalator ganha). Pra pequenos P ≈ 10-20% (N=200 cego é mais barato).

Solução próxima iteração: detectar tier do modelo e ajustar — modelos grandes usam (64, 100, 200), pequenos pulam pra (200,) direto. Não implementado ainda.

5. O que faltou

- **Validação real com modelo grande:** smoke usou Gemma-4B (modelo pequeno). Precisaria Qwen-Coder-32B ou Coder-Next pra ver parse_ok > 0 e behavior-modal-vote funcionando.
- **Per-model tier policy:** hoje todos os modelos usam 64,100,200. Pequenos deviam pular pra 200 direto.
- **Bench full:** 3 modelos × 12 cases × 5 lados rodando com escalator + schema. Estimativa ~8,900 chamadas, ~30-45min, ~\$2 OpenRouter.

6. Arquivos novos / modificados

Arquivo	Mudança	Linhas
bench/sp_output_schema.py	novo — schema v1 + StructuredResponse + behavior_modal_vote	+175
bench/sp_fanout_helper.py	adicionou sp_fanout_escalating() + ESCALATION_TIERS	+130
bench/run_exec_sindico.py	one_sp_fanout aceita oracle, passa pra escalator	+30
bench/run_offline.py	mesma mudança lado regex; oracle = score-all-match	+25
simplicio/ecosystem.py	novo — auto-upgrade ecosystem deps (commit anterior)	+210
simplicio/cli.py	hook maybe_run_session_start() na entrada	+10

Total: ~580 linhas novas + refator. Zero LLM call em testes (determinístico).

7. Próximo passo recomendado

Smoke focado com Qwen-Coder-32B (modelo grande) — 1 modelo × 3 cases × tiers 64,100,200 reais — ~600 chamadas no pior caso, ~10 min, < \$0.20 OpenRouter. Valida:

1. JSON schema funciona em modelo grande (parse_ok > 0)
2. Behavior modal-vote cria poucos grupos (não uniq=200)
3. Escalator para cedo quando oracle passa (cycle 1 fecha)
4. O ganho de custo se realiza na prática

Comando:

```
HF_TOKEN=... OPENROUTER_API_KEY=... \  
BENCH_SIMPLICIO_PROMPT_PATH=/tmp/prompt_check/prompts/agent-runtime-execution-prompt.md \  
BENCH_MODELS="qwen/qwen-2.5-coder-32b-instruct" \  
BENCH_INCLUDE_SP=1 BENCH_INCLUDE_AGENTS=0 BENCH_INCLUDE_SP_AG=0 \  
BENCH_SP_TIERS="64,100,200" \  
python3 bench/run_exec_sindico.py 2>&1 | tee /tmp/sp_validation.log
```

Se cycle 1 fechar $\geq 2/3$ dos cases, escalator está justificado. Senão, volta pra N=200 cego com per-model tier policy.